

The pipeline era is over.

Application teams declare intent.

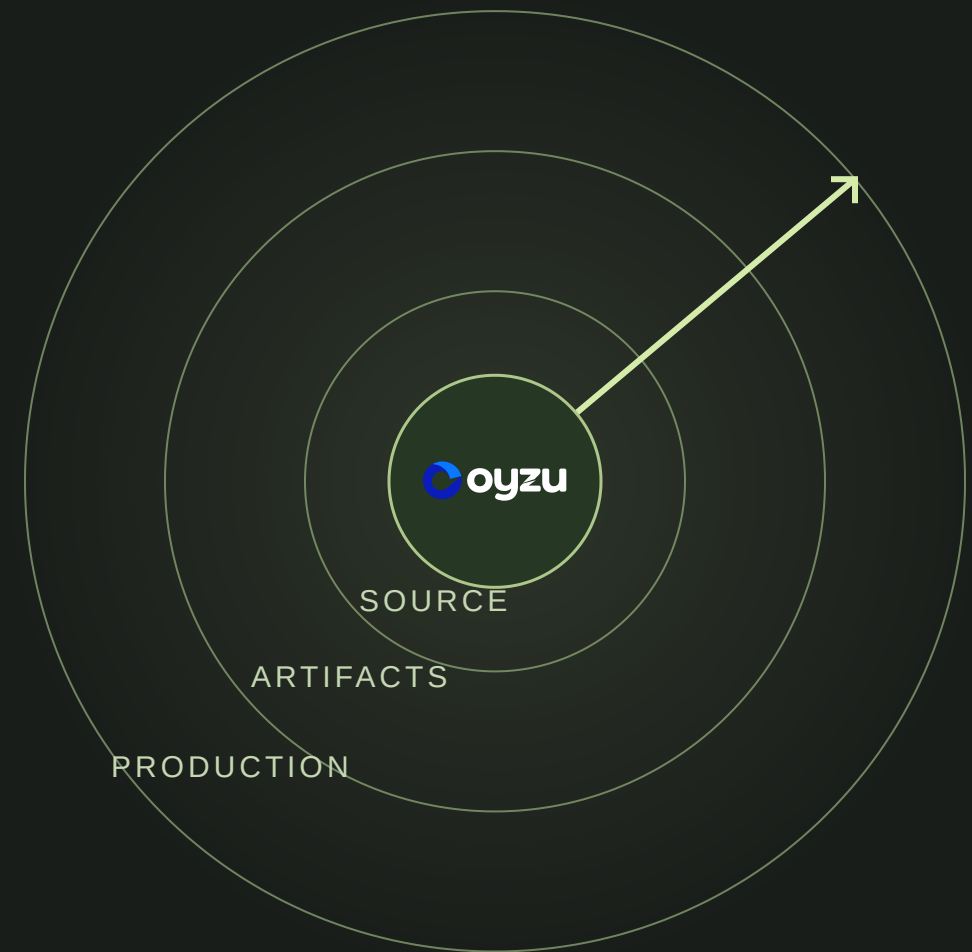
Organizations define policy.

Oyzu delivers.

\$1M

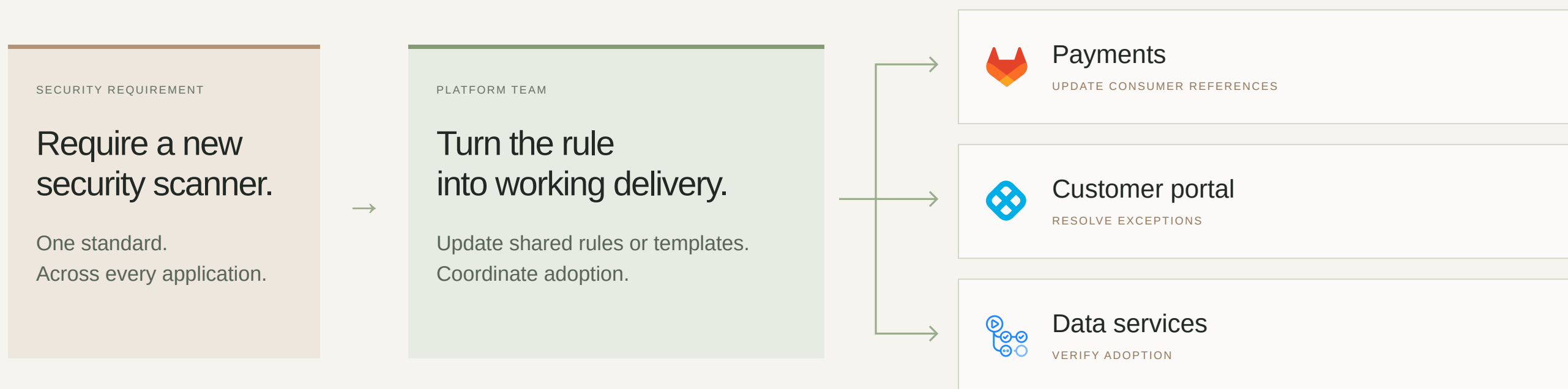
SEED ROUND

Source → artifact → production



ONE GOVERNED PATH · SOURCE TO PRODUCTION

One new requirement. Hundreds of workflows to change.



REPEAT ACROSS THE ESTATE

More applications → more owners → more exceptions to resolve

The requirement is shared.
The implementation work keeps multiplying.

Illustrative enterprise rollout; scale and effort vary. Shared templates and central policies reduce duplication but do not eliminate adoption and exception work.

More changes. More to govern.

MERGED PULL REQUESTS

98%

more merged pull requests.

Nearly twice the changes.

Faros's 2025 enterprise telemetry study compared periods of lower and higher AI adoption within teams.

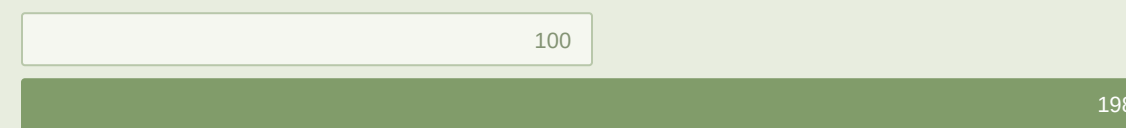
10,000+ developers

1,255 teams · observational study

LOWER-ADOPTION PERIOD = 100 · INDEXED METRICS

Merged pull requests

+98%



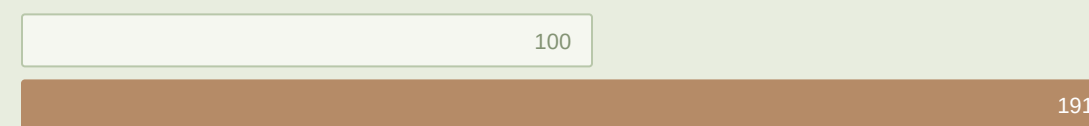
Average pull-request size

+154%



Pull-request review time

+91%



□ Lower AI adoption ■ Higher AI adoption

More output enters the delivery system.

The work to verify and govern it does not disappear.

“Can’t AI just write the pipelines?”

THE SHORT ANSWER

Writing it is only the beginning of owning it.

AI can help create and maintain a pipeline. The customer still owns the workflow and its obligations.

Validate

Update

Govern

Integrate

Collect evidence

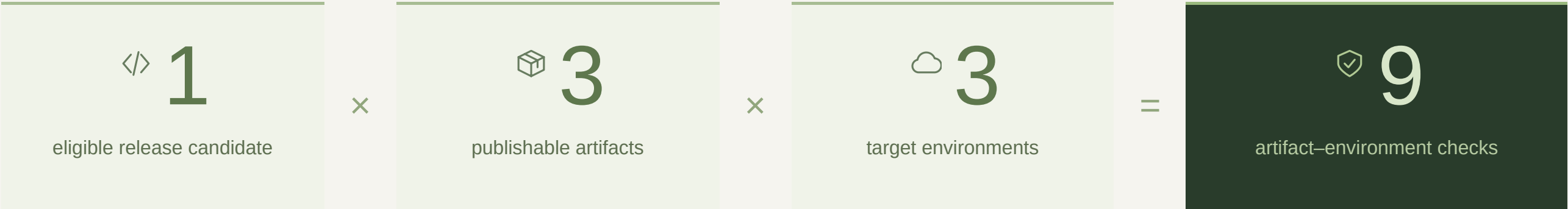
WHAT OYZU REMOVES

The application pipeline
your team has to own.

Teams declare intent. Organizations define policy.
Delivery expertise is built into the product.

No application pipelines to author. No pipeline templates to maintain.

One release. Many things to govern.



ILLUSTRATIVE MONTHLY LOAD · FIXED 3 × 3 SCOPE



Potential gate evaluations, not completed deployments or security incidents.

EACH ARTIFACT ALSO NEEDS

Identity. Checks. SBOM.
Provenance. Attestations.
Publication and deployment status.

More changes expand the governed workload—even before adding more targets or controls.

Oyzu scenario model, not an industry forecast. Assumes every candidate produces 3 artifacts and each is evaluated for 3 environments.

A dependency is compromised.

What is exposed? What can ship?



ILLUSTRATIVE SECURITY INCIDENT

Find affected releases. Change requirements. Verify the response.

DEPENDENCY ALERT

Package X
version A

Compromised

A component now
prohibited by policy.



CONNECTED EVIDENCE

payments-api · release A

SBOM identifies the component.
Artifact digest links to recorded deployments.

Affected release



AWS

Production deployment
Response required



GOVERN WHAT HAPPENS NEXT

Blocked

Further promotion of the affected artifact

Exception

Explicit owner, scope and expiry

Verified

A rebuilt candidate meets current requirements

Fragmented delivery turns response into an investigation.

Oyzu's target: trace exposure, apply policy, and retain proof of each decision.

Three responsibilities. One system.

01

APPLICATION TEAMS

Declare intent.

Describe the application and outcome.
Keep native project files.

“Deliver this service to staging.”

02

ORGANIZATIONS

Define policy.

Set approved tools, checks, evidence
and delivery conditions.

“Only approved, verified artifacts.”

03

OYZU

Deliver.

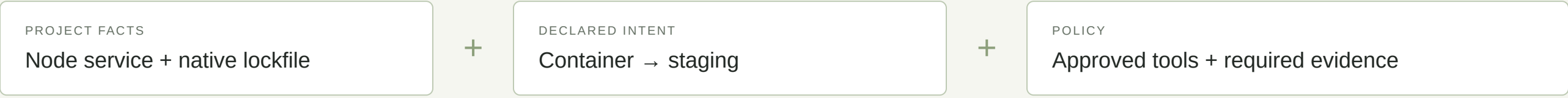
Generate the work. Execute the plan.
Retain proof of the outcome.

Source → artifact → production

Delivery expertise is built in. The enterprise stays in control.

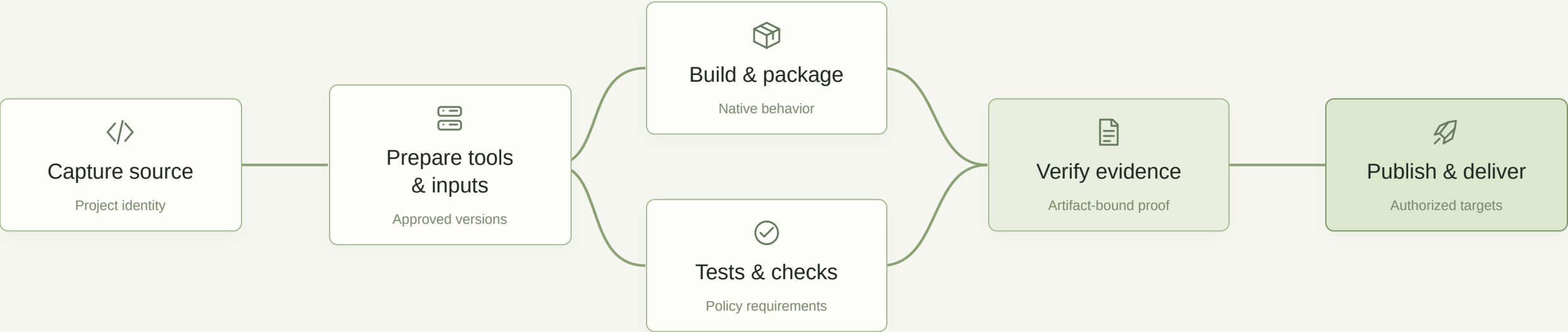
Intent defines the outcome. Policy defines the requirements. Oyzu generates the work.

Policy becomes executable.



↓ GENERATED EXECUTION PLAN

Derived per run · frozen before execution



+ NEW RULE

Require a license assessment

→

The next plan gains the assessment and its evidence gate.
No application pipeline edit.

Conceptual dependency graph; exact ordering depends on native behavior and policy.

Governance starts where code starts.

WHEN CONTROLS ARRIVE ONLY IN CI

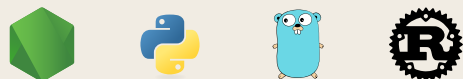
Choose tools → Build the feature → CI rejects the runtime

Now the team faces rework, exception requests and another approval queue.

THE OYZU EXPERIENCE

Make the approved path the easy path.

Repositories describe the application.
Oyzu supplies the delivery expertise.



AT EACH MANAGED INVOCATION

POLICY	Resolve governing policy centrally
RESOLVE	Select a policy-approved version
PREPARE	Acquire trusted tools and inputs
AUTHORIZE	Provide scoped dependency access

A conflict is explained before work builds on it.

Illustrative governed workflow. Controls apply to Oyzu-managed paths; broader endpoint enforcement requires additional controls. Implemented scope appears on the working-foundation slide.

One requirement. One governed rollout.

NEW STANDARD Require a license assessment before publication.

01 · CAPABILITY

Ship support.

Release the integration through Oyzu. Identify compatible CLI versions.



02 · VALIDATION

Pilot it.

Test a rollout group. Surface compatibility issues and exceptions.



03 · POLICY

Activate it.

Set scope, approved versions, deadlines and expiring exceptions centrally.



04 · ENFORCEMENT

Apply it.

Compatible clients generate the required work. Older clients receive upgrade requirements.

POLICY-CONTROLLED ADOPTION

Notify → Restrict publication → Require upgrade

Platform authorization gates managed credentials and publication.

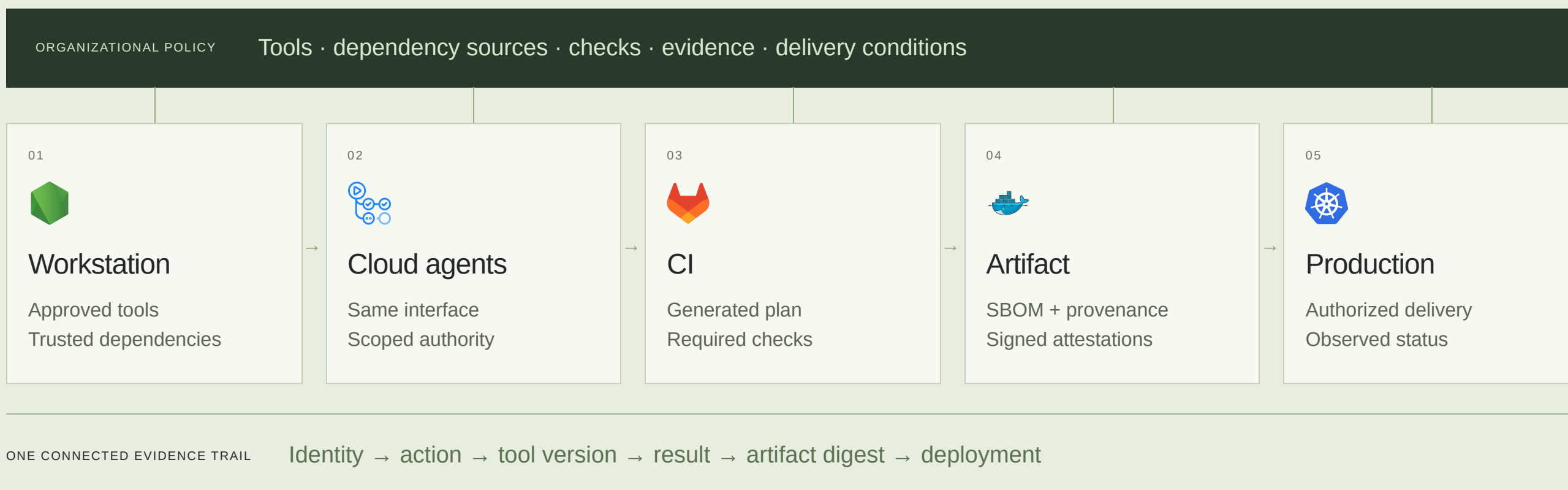
ONE MANAGED MECHANISM

Developer machines Cloud agents Existing CI

Roll out standards. Not repository migrations.

Illustrative rollout. New capabilities require compatible clients. Application changes, pilots and authorized exceptions may still be necessary.

One policy. From first install to production.



The integration is the product.

Oyzu manages policy distribution, credential coordination, evidence retention and verification.
Customers keep their existing execution and delivery infrastructure.

Architecture illustration; logos show ecosystem examples, not endorsements. Evidence records observed activity and identifies missing coverage.

The artifact is the record.

Every action. Every attestation. Every observed deployment.

payments-api / release B

Artifact detail

CONCEPT RECORD

ARTIFACT IDENTITY

One artifact.
Connected history.

SOURCE
Captured revision + tree

TOOLCHAIN
Runtime + builder identities

POLICY
Revision + resolved decisions

ARTIFACT
Content digest

SBOM

Provenance

Attestations

ACTIONS & OBSERVED STATUS · ILLUSTRATIVE

● Build completed

Native commands, logs and test results retained

Succeeded

● Release eligibility checked

Required evidence verified against current policy

Authorized

● Artifact published

Destination receipt bound to the artifact digest

Published

● Staging deployment observed

Deployment event links this release to its target

Deployed

● Production deployment

No deployment receipt has been reported

Not confirmed

One record across connected build systems and deployment destinations.

Missing evidence is visible—not assumed success.

Illustrative artifact record; displayed events and states are examples.

Shipping is only half the job. Then comes proving it.

REPORTED COMPLIANCE WORKLOAD

~10

hours / week

Policy reviews. Evidence collection.
Vendor attestations.

12 working weeks a year

Vanta's reported annual equivalent

Survey of business and IT leaders; not an engineering-only measure.

THE OYZU EXPERIENCE

Evidence is a delivery output.

01 Define the required controls

Versioned policy identifies required checks, approvals and evidence.

02 Capture proof as work happens

Bind actions, identities, results and attestations to the exact artifact.

03 Retrieve the record when asked

Trace source to deployment. Export control-linked evidence and expose gaps.

Less reconstruction at audit time. More traceability throughout delivery.

Who owns the delivery logic?

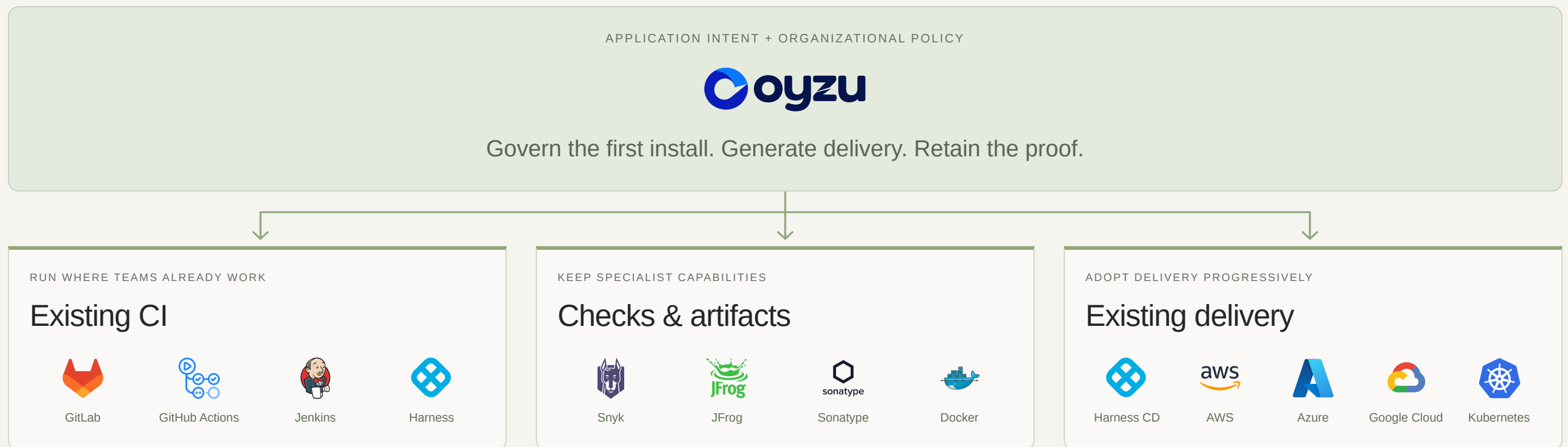
Powerful orchestration still leaves a workflow to own.
Oyzu makes generating that workflow the product’s responsibility.

	 GitLab	 Harness	 THE OYZU MODEL
Core approach	Integrated DevSecOps with CI/CD pipelines	Broad delivery orchestration with pipelines	Intent + native project facts + policy → execution plan
Reuse & governance	Auto DevOps, shared configuration and pipeline execution policies	Reusable pipeline / stage / step templates and policy as code	Policy governs generation; no application pipeline templates
Customer owns	Pipeline configuration or adopted defaults; customization and exceptions	Pipeline design or adopted templates; integrations and exceptions	Application intent, organizational rules and explicit exceptions
Where each is strong	Integrated source-to-delivery lifecycle	Deep delivery capabilities and orchestration flexibility	Governed tools from the workstation; product-owned delivery and connected evidence

The bet: less customer-owned automation to build, roll out and maintain.

[GitLab Auto DevOps](#) · [Pipeline policies](#) · [Harness templates](#) · [Policy as code](#). Operating-model comparison; implementation status on the working-foundation slide.

Keep the investment. Change the operating model.



Direct competition

Who owns application delivery logic, governance and the associated platform budget.

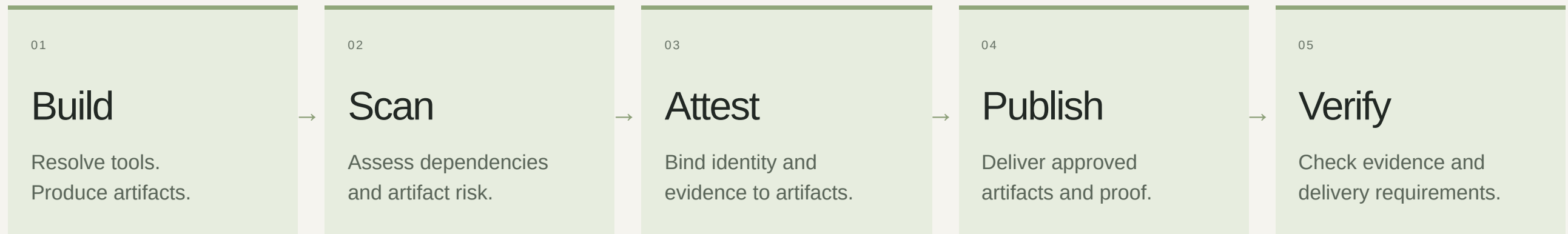
Better together, early

Use existing execution and specialist delivery. Make Oyzu adoption possible without a wholesale migration.

Intended integration strategy, not a list of shipped integrations or commercial partnerships. Existing CI/CD may retain a bootstrap or handoff workflow.

The full delivery lifecycle.

A working build foundation.



WORKING TODAY · DOCUMENTED NATIVE FIXTURES

Build behavior with retained evidence.

Passing tests retain artifacts and reports.

Assertion or coverage failures retain failure evidence.

Repeated qualified Yarn builds produced identical archive digests.

[Inspect the historical CI evidence →](#)

WHAT THIS ROUND ADVANCES

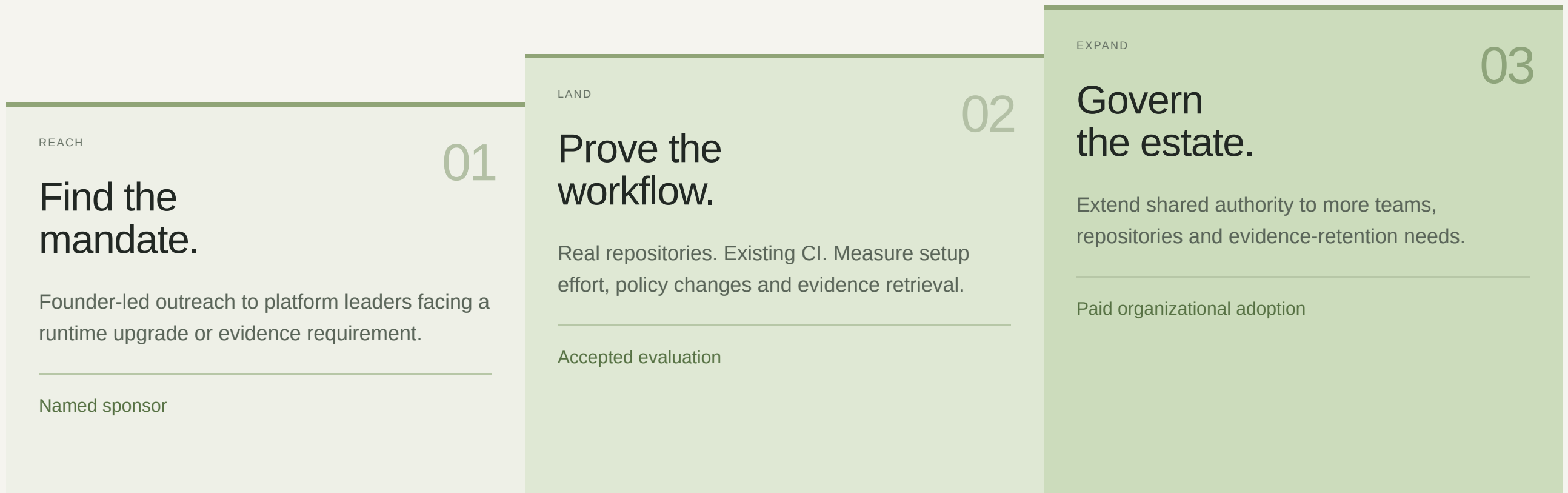
Managed, governed publication.

Central policy, evidence services and publication gates.

Broader delivery integrations build on that foundation.

One product owns the supported delivery mechanics.

One mandate. An organization-wide opportunity.



INITIAL BUYER Head of Platform Engineering / DevSecOps

Multiple application teams. Mixed CI. Recurring mandates.

Built for this problem.



FOUNDER

Micah Martin

Platform engineering leader.
Entrepreneur. Builder.

20 YEARS BUILDING
DEVELOPER INFRASTRUCTURE

BUILD

Engineering & DevOps leadership

Lifion by ADP · Engineering Manager, DevOps
SoulCycle · Director of Engineering

GOVERN

Enterprise operating experience

Current AVP, Platform Engineering · major enterprise

SELL

Customer-facing delivery expertise

Harness · Solutions Architecture → Director · 2020–2024

ADDITIONAL COFOUNDERS · DETAILS TO FOLLOW

Cofounder 2

Name · role · background to come

Cofounder 3

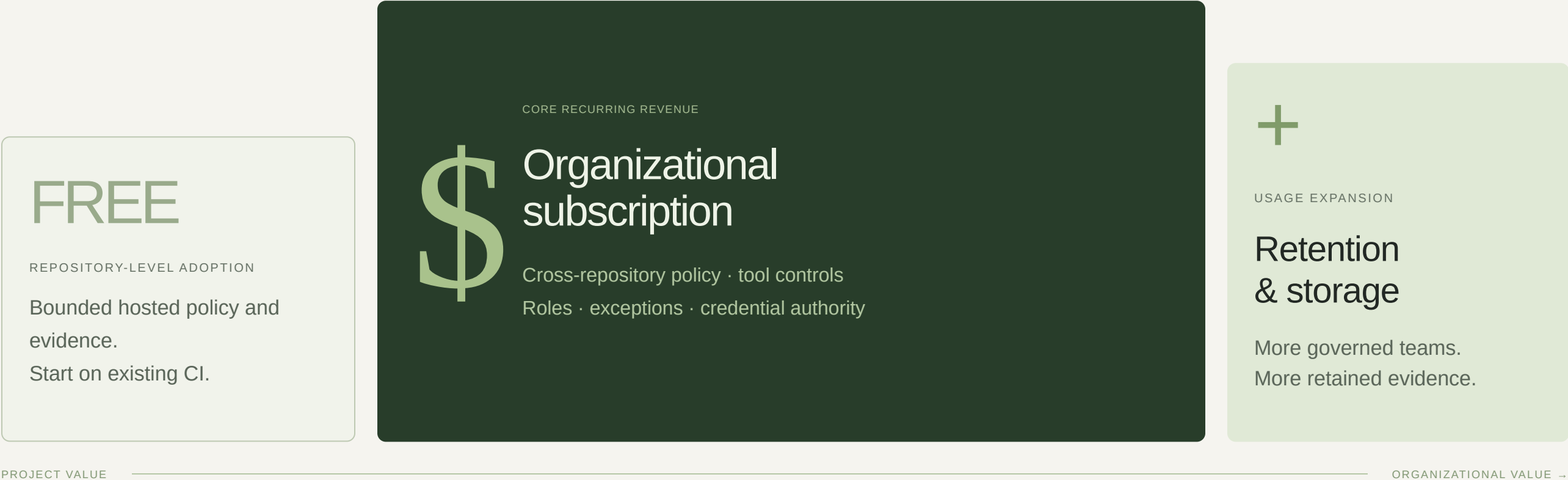
Name · role · background to come

Cofounder 4

Name · role · background to come

Role history: founder-provided LinkedIn screenshots. Career duration and entrepreneurship: founder-provided. Photo: GitHub @micahlmartin. Additional profiles are placeholders.

Adoption earns trust.
Governance earns revenue.



Existing execution first. Optional Oyzu compute becomes a separate usage business later.

COMMERCIAL MODEL · NOT A REVENUE FORECAST

Commercial model. Pricing, conversion, retention and service costs need validation; no revenue or margin forecast is asserted.

\$1M

To make governed delivery
a repeatable enterprise product.



- \$600K
Product & engineering
- \$250K
Customer validation & onboarding
- \$150K
Operations & reserve

3 Design partners

6 Independent repositories
across ≥ 2 CI systems

2 Paying organizations

ROUND EXIT TARGETS · ONBOARDING ≤ 1 ENGINEER-DAY PER SUPPORTED PROJECT

Funding targets and illustrative 12-month plan, not commitments or a guaranteed runway. Cost assumptions and acceptance measures are in the appendix.